



UNIVERSIDADE FEDERAL DO CEARÁ

RELATÓRIO: TRABALHO 2

Aluno:

Francisco Werley da Silva - 553948

Camile Isidório Araújo - 555251

Professor: Rafael Braga

Disciplina: Sistemas Distribuídos

1. Introdução

- O que aprenderam e o que ainda tem dificuldade?
 - Aprendemos melhor como funciona o RMI;
 - Sem muitas dificuldades.
- Nota que a dupla merece (em uma escala de 0 a 10);
 - 10
- Nível de dificuldade (muito fácil, fácil, média, difícil ou muito difícil);
 - média/difícil
- Quantas horas precisaram para completar a atividade? (Total = estudo pessoal + leituras + videoaulas extra + implementação)
 - 5 horas em média
- Caso desejem, deixem alguma sugestão para melhorar a atividade.
 - sem sugestões
- Qual o link do repositório com o código-fonte?
 - [LINK](#)

2. Descrição da implementação

Este trabalho apresenta o desenvolvimento de uma aplicação de Gestão de Recursos Humanos em Java, com foco em sistemas distribuídos e invocação remota de métodos. Diferentemente do trabalho anterior, em que a comunicação era feita por streams e sockets TCP, aqui a comunicação cliente-servidor foi implementada por meio de Java RMI (Remote Method Invocation), organizada em um protocolo de requisição-resposta conforme a seção 5.2 do livro-texto da disciplina.

A modelagem do domínio partiu da superclasse abstrata *Colaborador*, com atributos *id*, *nome* e *salário*, e das subclasses *Funcionario*, *Estagiario*, *Autonomo* e *Efetivo*, permitindo especialização e reaproveitamento por herança. A classe *Efetivo* estende *Funcionario* e adiciona bônus anual e anos de empresa, formando uma hierarquia de três níveis. A interface *Admissivel*, implementada por *Funcionario* e *Efetivo*, define o comportamento de admissão e permite polimorfismo na exibição de mensagens de admissão. A classe *Departamento* representa agregação: possui um gerente (*Colaborador*) e uma lista de colaboradores membros, além de calcular a folha somando o custo total de cada membro.

Cada tipo de colaborador implementa *calcularCustoTotal()* de forma distinta. *Funcionario* e *Efetivo* aplicam encargos de 68% sobre o salário; *Estagiario* soma vale-transporte fixo e seguro; *Autonomo* aplica acréscimo de 5%. Esse polimorfismo é usado tanto localmente quanto nas operações remotas de cálculo de folha.

A arquitetura foi dividida em quatro pacotes. O pacote `common` concentra entidades, `RemoteObjectRef`, a interface remota `RemoteService`, `MethodIds` e `JsonSerializer`. O pacote `protocol` contém `Message` e `RequestReplyProtocol`. O pacote `server` abriga `RHServer`, `ColaboradorServiceImpl` e `DepartamentoServiceImpl`. O pacote `client` contém `RHClient`, com menu interativo em modo texto.

A implementação remota usa Java RMI. O servidor `RHServer` instancia `ColaboradorServiceImpl` e `DepartamentoServiceImpl`, que estendem `UnicastRemoteObject` e implementam `RemoteService`. Em seguida cria o RMI Registry na porta 1099 e registra os objetos como "`ColaboradorService`" e "`DepartamentoService`". O cliente não possui cópia desses serviços: mantém apenas `RemoteObjectRef` (host, porta e nome do objeto) e obtém stubs via `LocateRegistry.getRegistry().lookup()`. Isso caracteriza passagem por referência para objetos remotos.

O protocolo request-reply foi implementado na classe `RequestReplyProtocol`. No cliente, `doOperation(RemoteObjectRef ref, int methodId, byte[] arguments)` monta uma `Message` do tipo `REQUEST` com `messageType`, `requestId`, `objectReference`, `methodId` e `arguments`, serializa em bytes com cabeçalho de tamanho, invoca `processRequest()` no stub remoto e desempacota a `REPLY` recebida. No servidor, `ColaboradorServiceImpl` e `DepartamentoServiceImpl` recebem o frame via RMI, chamam `getRequest()` para desempacotar a requisição, executam o método correspondente no repositório local e formatam a resposta com `sendReply()`. Com RMI, `sendReply` retorna o frame de resposta ao invocador remoto; o endereçamento IP/porta do cliente fica a cargo do runtime RMI, sem criação manual de sockets na aplicação.

As mensagens são empacotadas pela classe `Message`. O conteúdo é convertido para JSON e precedido por quatro bytes indicando o tamanho do payload. Os argumentos e resultados das operações também trafegam em JSON, por meio de `JsonSerializer`, caracterizando passagem por valor para objetos locais. Ao adicionar um colaborador, por exemplo, o cliente serializa o objeto para JSON, envia na requisição, e o servidor reconstrói o `Colaborador` antes de armazená-lo em `ConcurrentHashMap`.

Foram expostos dez métodos remotos, identificados por constantes em `MethodIds`. `ColaboradorService` oferece `adicionarColaborador`, `buscarColaborador`, `listarColaboradores`, `removerColaborador` e `calcularFolhaTotal`. `DepartamentoService` oferece `criarDepartamento`, `buscarDepartamento`, `listarDepartamentos`, `adicionarColaboradorAoDepartamento` e `calcularFolhaDepartamento`. `DepartamentoServiceImpl` depende de `ColaboradorServiceImpl` para resolver gerentes e membros por id, mantendo a lógica de negócio separada da modelagem de entidades.

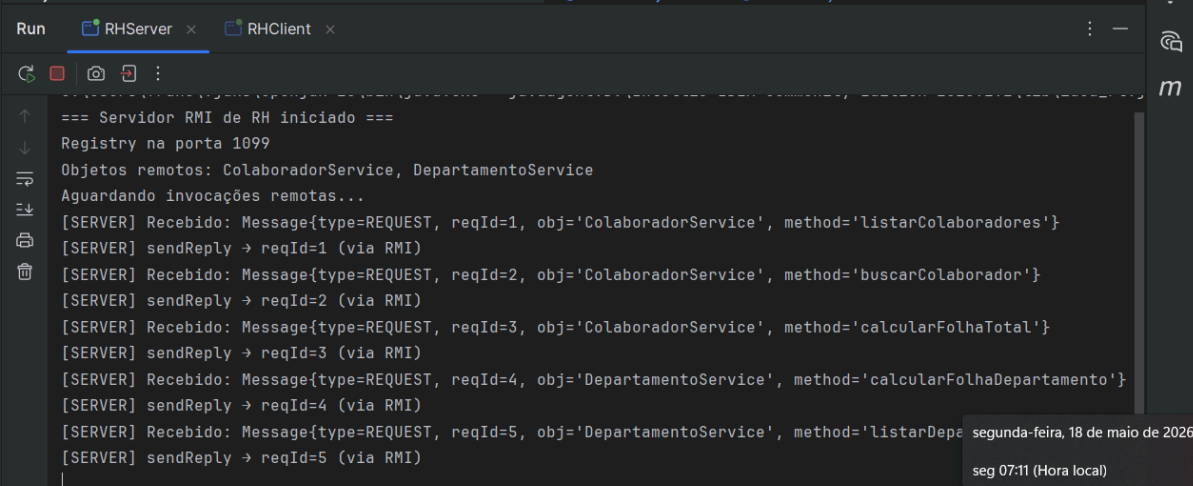
O cliente `RHClient` apresenta menu com dez opções. Cada operação monta os argumentos em JSON, chama `doOperation` com a referência remota e o `methodId` adequado, e converte a resposta JSON para exibição. Na busca de colaborador, se o resultado implementa `Admissivel`, a mensagem de `admitir()` também é exibida. O servidor inicia com dados de exemplo: quatro colaboradores e dois departamentos pré-cadastrados.

Para executar, compila-se com Maven (`mvn package`), inicia-se `RHServer` e, em outro terminal, `RHClient`. O servidor permanece aguardando invocações; o cliente conecta ao Registry e realiza as operações remotamente.

Como resultado, o trabalho consolidou a implementação prática de invocação remota, protocolo request-reply, passagem por referência e por valor, e representação externa de dados em JSON. A separação em camadas (domínio, protocolo, servidor e cliente) facilitou a organização e a manutenção. Em relação ao trabalho anterior com streams e sockets, o uso de RMI abstrai o transporte de rede, reduzindo a complexidade de conexão manual, mas exige compreensão de Registry, stubs, interfaces Remote e empacotamento de mensagens. A abordagem atende aos requisitos do enunciado: comunicação distribuída via RMI, protocolo request-reply da seção 5.2, modelagem OO com herança e agregação, e interface em modo texto.

Prints:

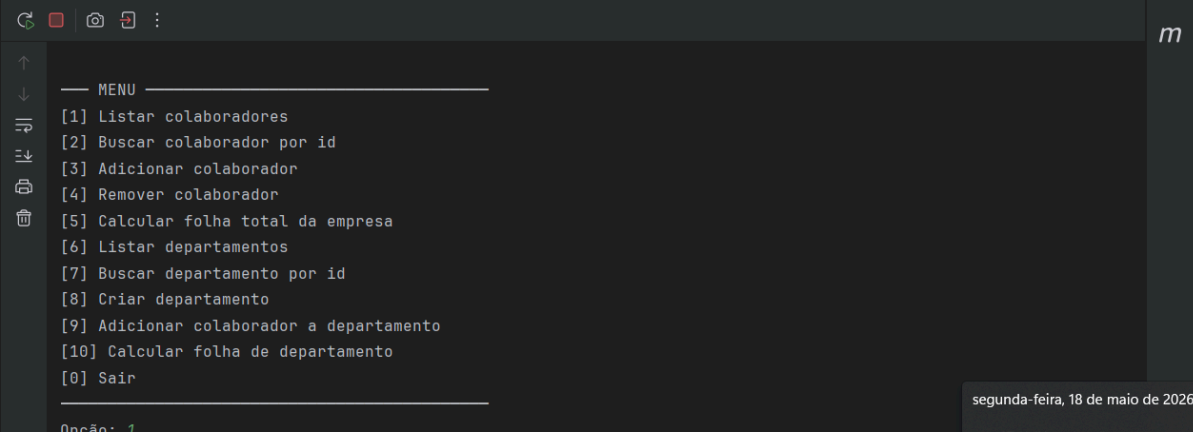
Servidor:



```
Run RHServer x RHClient x
=== Servidor RMI de RH iniciado ===
Registry na porta 1099
Objetos remotos: ColaboradorService, DepartamentoService
Aguardando invocações remotas...
[SERVER] Recebido: Message{type=REQUEST, reqId=1, obj='ColaboradorService', method='listarColaboradores'}
[SERVER] sendReply → reqId=1 (via RMI)
[SERVER] Recebido: Message{type=REQUEST, reqId=2, obj='ColaboradorService', method='buscarColaborador'}
[SERVER] sendReply → reqId=2 (via RMI)
[SERVER] Recebido: Message{type=REQUEST, reqId=3, obj='ColaboradorService', method='calcularFolhaTotal'}
[SERVER] sendReply → reqId=3 (via RMI)
[SERVER] Recebido: Message{type=REQUEST, reqId=4, obj='DepartamentoService', method='calcularFolhaDepartamento'}
[SERVER] sendReply → reqId=4 (via RMI)
[SERVER] Recebido: Message{type=REQUEST, reqId=5, obj='DepartamentoService', method='listarDepe
[SERVER] sendReply → reqId=5 (via RMI)
```

segunda-feira, 18 de maio de 2026
seg 07:11 (Hora local)

Cliente:



```
— MENU —
[1] Listar colaboradores
[2] Buscar colaborador por id
[3] Adicionar colaborador
[4] Remover colaborador
[5] Calcular folha total da empresa
[6] Listar departamentos
[7] Buscar departamento por id
[8] Criar departamento
[9] Adicionar colaborador a departamento
[10] Calcular folha de departamento
[0] Sair

Opção: 1
```

segunda-feira, 18 de maio de 2026
seg 07:11 (Hora local)